

API 명세서

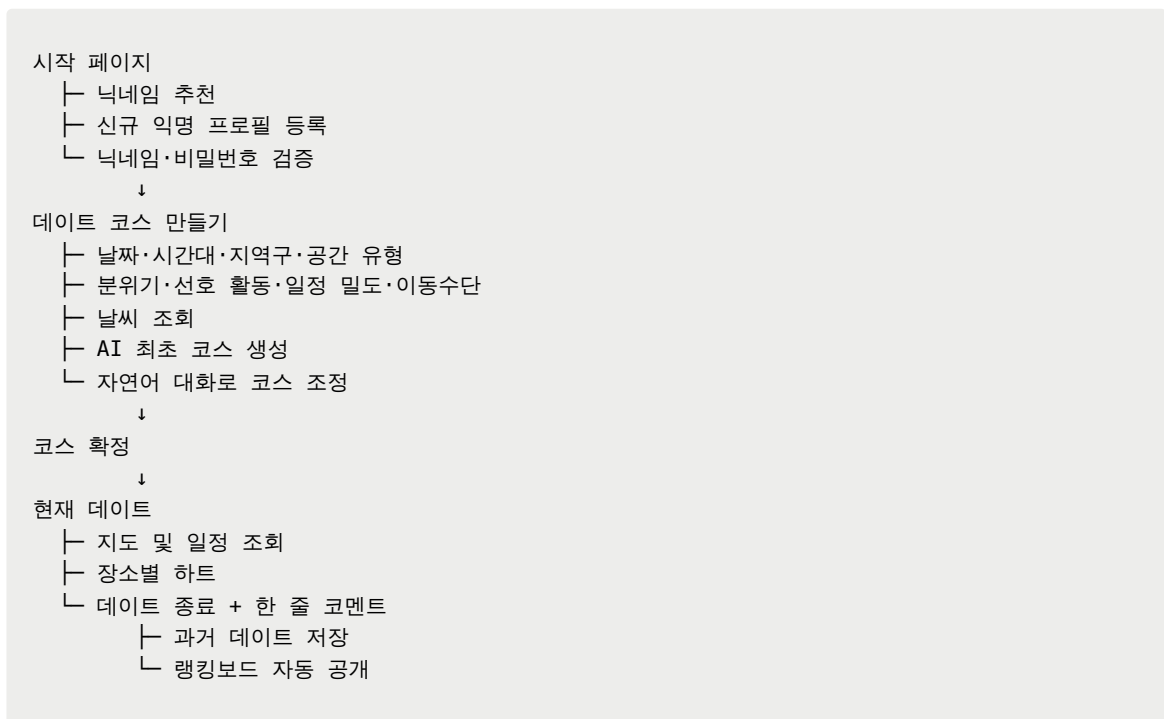
▼ v3.0

- 프로젝트: 사이42 — AI 기반 대전 데이트 메이커
 - 백엔드: FastAPI + SQLAlchemy + SQLite
 - API Prefix: `/api/v1`
 - 기본 Content-Type: `application/json`
 - 시간대: `Asia/Seoul`
 - 날짜 형식: `YYYY-MM-DD`
 - 일시 형식: ISO 8601 + timezone
 - ID 형식: UUID v4 문자열
 - 사용자 검증 방식: `X-Profile-Id` + `X-User-Password`
 - 응답 필드 표기: camelCase
 - DB 컬럼 표기: snake_case
-

▼ 0. 수정 이력

번호	변경 사항	반영 내용
1	비밀번호 평문 저장	<code>password_hash</code> 제거, <code>password</code> 컬럼에 숫자 4자리 평문 저장 및 직접 비교
2	닉네임·비밀번호 수정 페이지	<code>PATCH /profiles/me</code> , <code>PATCH /profiles/me/password</code> 삭제
3	성향 매번 조사	Preference 도메인·API·테이블 삭제, 코스 생성 요청 마다 전체 성향 조건 필수 전달
4	축제 조회 및 추천 반영 페이지	Festival 도메인·API·DTO·검증·추천 로직 모두 삭제
5	현재 데이트 취소 페이지	<code>DELETE /date-courses/current</code> 삭제, 활성 코스는 반드시 완료 후 새 코스 생성 가능

0.1 최종 사용자 흐름



0.2 익명 프로필 정책

익명 프로필

- 이메일, 실명, 전화번호를 수집하지 않는다.
- 닉네임과 숫자 4자리 비밀번호만 등록한다.

- 닉네임은 서비스 내에서 중복될 수 없다.
- 비밀번호는 교육용 요구사항에 따라 SQLite에 평문 저장한다.
- 서버는 Access Token, Refresh Token, Cookie Session을 발급하지 않는다.
- 보호 API는 요청마다 `profileId` 와 비밀번호를 검증한다.
- 닉네임과 비밀번호를 변경하는 기능은 제공하지 않는다.
- 비밀번호 분실 시 복구 기능도 제공하지 않는다.

| 평문 비밀번호 저장은 교육 목적의 의도된 설계이며 실제 운영 서비스에는 부적합하다.

0.3 완료 코스와 커뮤니티 게시물 분리

- `DateCourse` : 실제 데이트 코스 기록
- `CommunityPost` : 완료 코스를 랭킹보드에 공개하는 게시물

데이트 종료 시 두 리소스를 하나의 트랜잭션으로 처리한다.

- `DateCourse.status` : `ACTIVE` → `COMPLETED`
- 한 줄 코멘트와 만족도 저장
- `CommunityPost` 자동 생성
- 완료된 `DateCourse` 의 장소·시간·추천 문구는 수정 불가
- `CommunityPost` 의 한 줄 코멘트는 수정 가능
- 게시물을 삭제해도 과거 데이트 기록은 유지

▼ 1. 도메인 분리

도메인	책임	주요 엔티티
Common	공통 응답, 예외 처리, 페이징, 옵션, 요청 추적	BaseDto, ErrorResponseDto
Identity	익명 프로필 등록, 입장 검증, 요청별 비밀번호 검증	UserProfile
Place	공공데이터 기반 장소 조회 및 지도 데이터	Place
Weather	외부 날씨 조회 및 코스 추천용 요약	WeatherSnapshot
Chat	AI 대화 세션, 최초 코스 생성, 자연어 코스 조정	ChatSession, ChatMessage
CourseDraft	확정 전 AI 코스 초안	CourseDraft, CourseDraftPlace
DateCourse	현재·완료 데이트 관리, 장소 하트, 종료 처리	DateCourse, DateCoursePlace, PlaceHeart
Community	완료 코스 공개, 커뮤니티 CRUD, 코스 좋아요	CommunityPost, CommunityLike
Ranking	공개 코스 랭킹, 데이트 마스터 집계	RankingReadModel
History	본인의 완료 코스 조회 및 재진행	DateCourseReadModel

▼ 2. 상태 및 Enum

2.1 DateCourseStatus

값	설명
ACTIVE	현재 진행 중인 데이트
COMPLETED	종료되어 과거 데이트에 저장된 상태

CANCELED 상태는 사용하지 않는다.

한 프로파일은 ACTIVE 코스를 최대 1개만 가질 수 있다.

2.2 CourseSourceType

값	설명
AI_CHAT	AI 챗봇에서 생성·확정
RANKING_COPY	랭킹보드 공개 코스 복사
HISTORY_RESTART	본인의 과거 코스 재진행

2.3 ChatSessionStatus

값	설명
ACTIVE	대화 및 코스 수정 가능
CONFIRMED	현재 데이트로 확정
DISCARDED	확정 전 초안 폐기
EXPIRED	유효기간 만료

2.4 CommunityPostStatus

값	설명
PUBLISHED	랭킹보드 노출
DELETED	작성자가 삭제하여 미노출

2.5 선택 Enum

TimeSlot

- MORNING
- AFTERNOON
- FULL_DAY

District

- DONG_GU
- JUNG_GU
- SEO_GU
- YUSEONG_GU
- DAEDEOK_GU
- ANY

SpaceType

- INDOOR
- OUTDOOR
- ANY

Mood

- QUIET
- EMOTIONAL
- LIVELY
- SPECIAL

ActivityType

- TOURISM
- CULTURE_EXHIBITION
- FOOD
- SHOPPING
- WALK
- LEPORTS

축제·공연은 선택 항목과 추천 대상에서 제외한다.

ScheduleDensity

- RELAXED
- NORMAL
- TIGHT

Transportation

- WALK
- PUBLIC_TRANSIT
- CAR
- FLEXIBLE

PlaceCategory

- ATTRACTION
- CULTURAL_FACILITY
- TRAVEL_COURSE
- LEPORTS
- ACCOMMODATION
- SHOPPING
- RESTAURANT

RankingSort

- OVERALL
 - HEARTS
 - LATEST
 - USAGE
-

▼ 3. 공통 응답 구조

3.1 BaseDto

모든 성공 응답은 아래 구조로 통일한다.

```
{
  "success": true,
  "code": "COMMON_OK",
  "message": "요청이 성공적으로 처리되었습니다.",
  "data": {},
  "meta": null,
  "timestamp": "2026-07-14T14:30:00+09:00",
  "traceId": "9bb43f9d-a680-42ca-93bf-8a87a91aaef2"
}
```

필드	타입	필수	설명
success	boolean	Y	성공 응답은 항상 true
code	string	Y	애플리케이션 응답 코드
message	string	Y	사용자 표시 가능한 메시지
data	T null	Y	실제 응답 데이터
meta	PageMetaDto null	Y	페이징 등 부가 정보
timestamp	datetime	Y	서버 응답 시각
traceId	UUID string	Y	로그 추적용 요청 ID

3.2 PageMetaDto

```
{
  "page": 1,
  "size": 20,
  "totalElements": 57,
  "totalPages": 3,
  "hasNext": true,
  "hasPrevious": false
}
```


페이지 번호는 1부터 시작한다.

3.3 ErrorResponseDto

```
{
  "success": false,
  "code": "AUTH_INVALID_CREDENTIALS",
  "message": "사용자 정보 또는 비밀번호가 올바르지 않습니다.",
  "errors": [
    {
      "field": null,
      "reason": "INVALID_CREDENTIALS",
      "rejectedValue": null
    }
  ],
  "timestamp": "2026-07-14T14:31:00+09:00",
  "traceId": "b4cd1da0-3c6c-4e80-a4ea-d74f0edc46e1"
}
```

필드	타입	필수	설명
success	boolean	Y	실패 응답은 항상 false
code	string	Y	도메인별 에러 코드
message	string	Y	사용자 표시용 메시지
errors	ErrorDetailDto[]	Y	상세 오류. 없으면 빈 배열
timestamp	datetime	Y	서버 응답 시각
traceId	UUID string	Y	로그 추적 ID

3.4 ErrorDetailDto

필드	타입	설명
field	string null	오류가 발생한 요청 필드
reason	string	기계 판독 가능한 원인
rejectedValue	any null	거부된 값

비밀번호는 rejectedValue 에 절대 포함하지 않는다.

3.5 FastAPI Pydantic 공통 모델

```

from datetime import datetime
from typing import Generic, TypeVar
from pydantic import BaseModel, ConfigDict, Field

T = TypeVar("T")

def to_camel(value: str) -> str:
    parts = value.split("_")
    return parts[0] + "".join(word.capitalize() for word in parts[1:])

class ApiModel(BaseModel):
    model_config = ConfigDict(
        alias_generator=to_camel,
        populate_by_name=True,
        from_attributes=True,
    )

class PageMetaDto(ApiModel):
    page: int
    size: int
    total_elements: int
    total_pages: int
    has_next: bool
    has_previous: bool

class BaseDto(ApiModel, Generic[T]):
    success: bool = True
    code: str
    message: str
    data: T | None = None
    meta: PageMetaDto | None = None
    timestamp: datetime
    trace_id: str

class ErrorDetailDto(ApiModel):
    field: str | None = None
    reason: str
    rejected_value: object | None = None

```

```
class ErrorResponseDto(ApiModel):
    success: bool = False
    code: str
    message: str
    errors: list[ErrorDetailDto] = Field(default_factory=list)
    timestamp: datetime
    trace_id: str
```

▼ 4. 공통 요청 규칙

4.1 사용자 검증 헤더

보호 API는 아래 헤더를 요구한다.

```
X-Profile-Id: 88a6f640-72fb-4e67-a05f-47b2809b76ff
X-User-Password: 0420
```

헤더	타입	필수	설명
X-Profile-Id	UUID string	보호 API Y	익명 프로필 식별자
X-User-Password	string	보호 API Y	숫자 4자리 비밀번호
X-Request-Id	UUID string	N	요청 추적 ID
Idempotency-Key	UUID string	일부 POST 권장	중복 실행 방지

4.2 검증 순서

1. 두 헤더 존재 여부 확인
2. X-Profile-Id UUID 형식 확인
3. 해당 프로필 조회
4. X-User-Password 숫자 4자리 형식 확인
5. DB의 평문 password 와 직접 비교
6. 성공 시 current_profile 주입
7. 소유 리소스 접근 시 소유자 ID 확인

4.3 검증 오류

상황	HTTP	code
두 헤더 중 하나 누락	401	AUTH_CREDENTIALS_REQUIRED
profileId 형식 오류	401	AUTH_INVALID_PROFILE_ID_FORMAT
프로필 없음	401	AUTH_INVALID_CREDENTIALS
비밀번호 불일치	401	AUTH_INVALID_CREDENTIALS
반복 실패 제한	429	AUTH_TOO_MANY_ATTEMPTS
타 사용자 리소스 접근	403	AUTH_FORBIDDEN

프로필 존재 여부와 비밀번호 오류를 구분해 노출하지 않는다.

4.4 공개 API의 선택 검증

장소, 날씨, 랭킹 조회는 헤더 없이 호출 가능하다.

- 헤더 둘 다 없음: 익명 조회
- 헤더 둘 다 있음: 검증 후 `likedByMe`, `owner` 등 개인화 정보 포함
- 하나만 있음: 401
- 둘 다 있으나 불일치: 401

4.5 비밀번호 전달 정책

- HTTPS 사용 필수
- 비밀번호를 URL Path 또는 Query String에 넣지 않음
- 서버 access log에서 `X-User-Password` 마스킹
- 프론트 `localStorage` 저장 금지
- Pinia 메모리 또는 `sessionStorage` 사용
- 페이지를 닫거나 퇴장 시 제거
- `profileId`와 IP 기준 rate limit 적용

4.6 FastAPI Dependency

```
from typing import Annotated
from uuid import UUID
from fastapi import Depends, Header
from sqlalchemy.ext.asyncio import AsyncSession

async def get_current_profile(
    x_profile_id: Annotated[UUID, Header(alias="X-Profile-Id")],
    x_user_password: Annotated[str, Header(alias="X-User-Password")],
    db: AsyncSession = Depends(get_db),
) -> UserProfile:
    profile = await profile_repository.find_by_id(db, x_profile_id)
```

```

    if profile is None or profile.password != x_user_password:
        raise BusinessException(
            status_code=401,
            code="AUTH_INVALID_CREDENTIALS",
            message="사용자 정보 또는 비밀번호가 올바르지 않습니다.",
        )

    return profile

CurrentProfile = Annotated[UserProfile, Depends(get_current_profile)]

```

4.7 CORS

```

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins,
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=[
        "Content-Type",
        "X-Profile-Id",
        "X-User-Password",
        "X-Request-Id",
        "Idempotency-Key",
    ],
)

```

4.8 멍등성

다음 API는 `Idempotency-Key` 사용을 권장한다.

- 코스 초안 확정
- 데이트 종료
- 랭킹 코스 진행

- 과거 코스 재진행
- 커뮤니티 재발행

동일 profileId, URL, Idempotency-Key 요청은 최초 결과를 반환한다.

4.9 문자열 제한

필드	제한
닉네임	2~14자
비밀번호	숫자 4자리
AI 대화 메시지	1~1000자
코스 제목	1~60자
한 줄 코멘트	1~80자
전체 추천 멘트	최대 500자
장소별 추천 멘트	최대 300자

▼ 5. API 전체 목록

Common

Method	URL	설명
GET	<code>/api/v1/health</code>	서버 상태 확인
GET	<code>/api/v1/meta/options</code>	코스 생성 선택 옵션 조회

Identity

Method	URL	설명
GET	<code>/api/v1/auth/nickname-suggestions</code>	재미있는 닉네임 추천
POST	<code>/api/v1/auth/profiles</code>	익명 프로필 등록
POST	<code>/api/v1/auth/verify</code>	닉네임·비밀번호 입장 검증
GET	<code>/api/v1/profiles/me</code>	내 프로필 기본 정보 조회

닉네임·비밀번호 수정 API는 제공하지 않는다.

Place / Weather

Method	URL	설명
GET	<code>/api/v1/places</code>	장소 목록·지도 마커 조회
GET	<code>/api/v1/places/{placeId}</code>	장소 상세 조회
GET	<code>/api/v1/places/{placeId}/nearby</code>	주변 장소 조회
GET	<code>/api/v1/weather</code>	날짜·지역 기반 날씨 조회

축제 조회 API는 제공하지 않는다.

Chat / CourseDraft

Method	URL	설명
POST	<code>/api/v1/chat/sessions</code>	매번 성향 조건을 받아 채팅 세션과 최초 코스 생성
GET	<code>/api/v1/chat/sessions/{sessionId}</code>	채팅 내역과 현재 코스 초안 조회
POST	<code>/api/v1/chat/sessions/{sessionId}/messages</code>	자연어로 코스 조정

Method	URL	설명
POST	<code>/api/v1/chat/sessions/{sessionId}/confirm</code>	코스 초안을 현재 데이터로 확정
DELETE	<code>/api/v1/chat/sessions/{sessionId}</code>	확정 전 코스 초안 폐기

Current DateCourse

Method	URL	설명
GET	<code>/api/v1/date-courses/current</code>	현재 데이트 조회
PUT	<code>/api/v1/date-courses/current/places/{coursePlaceId}/heart</code>	현재 장소 하트 등록
DELETE	<code>/api/v1/date-courses/current/places/{coursePlaceId}/heart</code>	현재 장소 하트 취소
POST	<code>/api/v1/date-courses/current/complete</code>	데이트 종료 및 자동 공개

현재 데이트 취소 API는 제공하지 않는다.

Community / Ranking

Method	URL	설명
GET	<code>/api/v1/community/posts</code>	랭킹보드 목록
GET	<code>/api/v1/community/posts/{postId}</code>	드롭다운 상세
POST	<code>/api/v1/community/posts</code>	완료 코스 커뮤니티 재발행
PATCH	<code>/api/v1/community/posts/{postId}</code>	한 줄 코멘트 수정
DELETE	<code>/api/v1/community/posts/{postId}</code>	커뮤니티 게시물 삭제
PUT	<code>/api/v1/community/posts/{postId}/like</code>	코스 좋아요
DELETE	<code>/api/v1/community/posts/{postId}/like</code>	코스 좋아요 취소
POST	<code>/api/v1/community/posts/{postId}/start</code>	공개 코스를 현재 데이터로 복사
GET	<code>/api/v1/rankings/masters</code>	데이트 마스터 랭킹

History

Method	URL	설명
GET	<code>/api/v1/profiles/me/date-courses</code>	과거 데이트 목록
GET	<code>/api/v1/profiles/me/date-courses/{courseId}</code>	과거 데이트 상세

Method	URL	설명
POST	<code>/api/v1/profiles/me/date-courses/{courseId}/restart</code>	과거 코스 다시 진행

▼ 6. 공통 도메인 DTO

6.1 PlaceSummaryDto

```
{
  "placeId": "e9428c38-4f2f-4f21-bbb9-d8049b9c6025",
  "contentId": "130551",
  "name": "대전시립미술관",
  "category": "CULTURAL_FACILITY",
  "district": "SEO_GU",
  "address": "대전광역시 서구 둔산대로 155",
  "latitude": 36.366988348,
  "longitude": 127.385704504,
  "imageUrl": "https://...",
  "indoorOutdoor": "INDOOR"
}
```

6.2 CourseConditionDto

코스 생성 시 매번 전달되는 성향·조건이다.

```
{
  "date": "2026-07-14",
  "timeSlot": "AFTERNOON",
  "startTime": "14:00",
  "district": "YUSEONG_GU",
  "spaceType": "INDOOR",
  "moods": ["QUIET", "EMOTIONAL"],
  "activities": ["CULTURE_EXHIBITION", "FOOD"],
  "scheduleDensity": "RELAXED",
  "transportation": "PUBLIC_TRANSIT"
}
```

서버는 해당 조건을 사용자 프로필 성향으로 저장하지 않는다.

6.3 WeatherSummaryDto

```
{
  "available": true,
  "district": "YUSEONG_GU",
  "date": "2026-07-14",
  "timeSlot": "AFTERNOON",
  "summary": "오후 4시부터 약한 비",
  "temperatureMin": 21.0,
  "temperatureMax": 27.0,
  "precipitationProbability": 60,
  "condition": "LIGHT_RAIN",
  "recommendedSpaceType": "INDOOR",
  "message": "비 예보를 반영해 실내 장소 비중을 높였습니다."
}
```

6.4 CoursePlaceDto

```
{
  "coursePlaceId": "f2a9252d-14b1-4fe4-ae09-c8ebac4fd4c2",
  "order": 1,
  "scheduledAt": "2026-07-14T14:00:00+09:00",
  "estimatedStayMinutes": 90,
  "place": {},
  "sweetComment": "같은 작품을 바라보며 서로의 취향을 조금 더 알아가 보세요.",
  "heartedByMe": false,
  "heartCount": 14
}
```

6.5 CourseMapDto

```
{
  "centerLatitude": 36.37,
  "centerLongitude": 127.38,
  "polyline": [
    {
      "latitude": 36.3798,
      "longitude": 127.3605
    }
  ]
}
```

```

    },
    {
      "latitude": 36.3669,
      "longitude": 127.3857
    }
  ]
}

```

6.6 DateCourseDto

```

{
  "courseId": "2adb4617-cf36-40ba-a6bd-2e9d45d2b3d8",
  "status": "ACTIVE",
  "sourceType": "AI_CHAT",
  "title": "비 오는 날 포근한 문화 데이트",
  "date": "2026-07-14",
  "timeSlot": "AFTERNOON",
  "overallComment": "오늘의 비 예보를 반영해 실내 장소 중심으로  
구성했어요.",
  "conditions": {},
  "tags": [
    "#유성구",
    "#오후데이트",
    "#실내데이트",
    "#문화전시",
    "#조용한분위기",
    "#넉넉한코스"
  ],
  "weather": {},
  "places": [],
  "map": {},
  "oneLineComment": null,
  "rating": null,
  "createdAt": "2026-07-14T13:30:00+09:00",
  "completedAt": null
}

```

▼ 7. Common API

7.1 Health Check

GET /api/v1/health

사용자 검증: 불필요

Response 200

```
{
  "success": true,
  "code": "COMMON_HEALTHY",
  "message": "서비스가 정상 동작 중입니다.",
  "data": {
    "status": "UP",
    "database": "UP",
    "aiProvider": "UP",
    "weatherProvider": "UP",
    "version": "3.0.0"
  },
  "meta": null,
  "timestamp": "2026-07-14T14:00:00+09:00",
  "traceId": "...
}
```

외부 API 장애 시 서버와 DB가 정상이면 HTTP 200을 유지하고 해당 Provider를 **DEGRADED** 로 표시한다.

7.2 선택 옵션 조회

GET /api/v1/meta/options

사용자 검증: 불필요

Response 200 data

```
{
  "timeSlots": [
```

```
    {"value": "MORNING", "label": "오전"},
    {"value": "AFTERNOON", "label": "오후"},
    {"value": "FULL_DAY", "label": "오전+오후"}
  ],
  "districts": [
    {"value": "DONG_GU", "label": "동구"},
    {"value": "JUNG_GU", "label": "중구"},
    {"value": "SEO_GU", "label": "서구"},
    {"value": "YUSEONG_GU", "label": "유성구"},
    {"value": "DAEDEOK_GU", "label": "대덕구"},
    {"value": "ANY", "label": "상관없음"}
  ],
  "spaceTypes": [],
  "moods": [],
  "activityTypes": [],
  "scheduleDensities": [],
  "transportations": [],
  "rankingSorts": []
}
```

▼ 8. Identity API

8.1 닉네임 추천

GET /api/v1/auth/nickname-suggestions

사용자 검증: 불필요

Query

필드	타입	필수	기본값	설명
count	int	N	5	1~10
seed	string	N	null	추천 분위기 키워드

Response 200 data

```
{
  "suggestions": [
    "복숭아와호두",
    "성심당옆두근이",
    "042설렘단",
    "구름한스폰",
    "대전한바퀴"
  ]
}
```

8.2 익명 프로필 등록

POST /api/v1/auth/profiles

사용자 검증: 불필요

Request

```
{
  "nickname": "복숭아와호두",
}
```



```
"password": "0420"
}
```

필드	타입	필수	검증
nickname	string	Y	2~14자, trim, 금칙어 제외
password	string	Y	정규식 <code>^\d{4}\$</code>

처리

1. 닉네임 정규화
2. 중복 확인
3. 비밀번호를 암호화하지 않고 평문 저장
4. 프로필 생성
5. profileId 반환

Response 201 data

```
{
  "profileId": "88a6f640-72fb-4e67-a05f-47b2809b76ff",
  "nickname": "복숭아와호두",
  "hasActiveDateCourse": false,
  "completedDateCourseCount": 0,
  "createdAt": "2026-07-14T10:00:00+09:00"
}
```

Errors

HTTP	code	조건
409	<code>AUTH_NICKNAME_ALREADY_EXISTS</code>	닉네임 중복
422	<code>COMMON_VALIDATION_ERROR</code>	형식 오류
429	<code>AUTH_TOO_MANY_ATTEMPTS</code>	반복 등록 시도

8.3 입장 검증

POST `/api/v1/auth/verify`

사용자 검증: Request Body로 수행

Request

```
{
  "nickname": "복숭아와호두",
  "password": "0420"
}
```

처리

1. 닉네임으로 프로필 조회
2. 저장된 평문 비밀번호와 직접 비교
3. 성공 시 profileId 반환
4. 토큰·쿠키·세션을 생성하지 않음

Response 200 data

```
{
  "profileId": "88a6f640-72fb-4e67-a05f-47b2809b76ff",
  "nickname": "복숭아와호두",
  "hasActiveDateCourse": true,
  "completedDateCourseCount": 3,
  "verifiedAt": "2026-07-14T10:01:00+09:00"
}
```

Errors

HTTP	code	설명
401	<code>AUTH_INVALID_CREDENTIALS</code>	닉네임 또는 비밀번호 불일치
429	<code>AUTH_TOO_MANY_ATTEMPTS</code>	요청 제한 초과

8.4 내 프로필 조회

GET /api/v1/profiles/me

사용자 검증: 필수

Response 200 data

```
{
  "profileId": "88a6f640-72fb-4e67-a05f-47b2809b76ff",
  "nickname": "복숭아와호두",
  "hasActiveDateCourse": true,
  "completedDateCourseCount": 3,
  "publishedCourseCount": 3,
  "createdAt": "2026-07-14T10:00:00+09:00"
}
```

닉네임과 비밀번호 수정 API는 존재하지 않는다.

▼ 9. Place API

9.1 장소 목록 및 지도 마커 조회

GET /api/v1/places

사용자 검증: 선택

Query

필드	타입	필수	설명
<code>keyword</code>	string	N	장소명·주소 검색
<code>district</code>	District	N	<code>ANY</code> 제외
<code>category</code>	PlaceCategory[]	N	썸표 구분
<code>spaceType</code>	SpaceType	N	실내·실외
<code>latitude</code>	float	N	중심 위도
<code>longitude</code>	float	N	중심 경도
<code>radiusKm</code>	float	N	0.1~30
<code>hasImage</code>	boolean	N	이미지 존재 여부
<code>page</code>	int	N	기본 1
<code>size</code>	int	N	기본 20, 최대 100

`latitude`, `longitude`, `radiusKm` 는 세 필드를 함께 전달해야 한다.

Response 200

`data` : `PlaceSummaryDto[]`

`meta` : `PageMetaDto`

Errors

- `PLACE_INVALID_COORDINATES`
- `PLACE_RADIUS_OUT_OF_RANGE`
- `COMMON_VALIDATION_ERROR`

9.2 장소 상세 조회

GET /api/v1/places/{placeId}

사용자 검증: 선택

Response 200 data

```
{
  "placeId": "...",
  "contentId": "130551",
  "contentType": "14",
  "name": "대전시립미술관",
  "category": "CULTURAL_FACILITY",
  "district": "SEO_GU",
  "address": "대전광역시 서구 둔산대로 155",
  "detailAddress": "",
  "latitude": 36.366988348,
  "longitude": 127.385704504,
  "imageUrl": "https://...",
  "thumbnailUrl": "https://...",
  "telephone": null,
  "indoorOutdoor": "INDOOR",
  "source": {
    "provider": "한국관광공사",
    "license": "공공누리 제3유형",
    "originalModifiedAt": "2026-05-29T10:23:29+09:00"
  }
}
```

9.3 주변 장소 조회

GET /api/v1/places/{placeId}/nearby

사용자 검증: 선택

Query

필드	타입	기본값
radiusKm	float	3
category	PlaceCategory[]	null
limit	int	10, 최대 30

Response 200 data

```
{
  "origin": {},
  "places": [
    {
      "place": {},
      "distanceKm": 1.2
    }
  ]
}
```

▼ 10. Weather API

10.1 날짜·지역 날씨 조회

GET /api/v1/weather

사용자 검증: 선택

Query

필드	타입	필수	설명
date	date	Y	공급자 예보 범위 내 날짜
district	District	Y	ANY 가능
timeSlot	TimeSlot	Y	오전·오후·하루

Response 200 data

```
{
  "available": true,
  "district": "YUSEONG_GU",
  "date": "2026-07-14",
  "timeSlot": "AFTERNOON",
  "summary": "오후 4시부터 약한 비",
  "temperatureMin": 21.0,
  "temperatureMax": 27.0,
  "precipitationProbability": 60,
  "condition": "LIGHT_RAIN",
  "recommendation": {
    "preferredSpaceType": "INDOOR",
    "indoorRatio": 0.7,
    "message": "오후 비 예보를 반영해 실내 장소를 중심으로 추천할  
게요."
  },
  "provider": "외부 날씨 API",
  "observedAt": "2026-07-14T10:00:00+09:00"
}
```

외부 API 실패 Fallback

HTTP 200을 유지한다.

```
{
  "available": false,
  "district": "YUSEONG_GU",
  "date": "2026-07-14",
  "timeSlot": "AFTERNOON",
  "summary": null,
  "recommendation": {
    "preferredSpaceType": "ANY",
    "indoorRatio": null,
    "message": "날씨 정보를 불러오지 못해 입력한 공간 조건만 반영합니다."
  }
}
```

▼ 11. Chat / CourseDraft API

11.1 채팅 세션 및 최초 코스 생성

POST /api/v1/chat/sessions

사용자 검증: 필수

사용자는 코스를 만들 때마다 성향과 조건을 새로 입력해야 한다.

Request

```
{
  "date": "2026-07-14",
  "timeSlot": "AFTERNOON",
  "startTime": "14:00",
  "district": "YUSEONG_GU",
  "spaceType": "INDOOR",
  "moods": ["QUIET", "EMOTIONAL"],
  "activities": ["CULTURE_EXHIBITION", "FOOD"],
  "scheduleDensity": "RELAXED",
  "transportation": "PUBLIC_TRANSIT",
  "initialMessage": "마지막에는 조용한 저녁 식당을 넣어줘."
}
```

필드	타입	필수	검증
date	date	Y	과거 날짜 불가
timeSlot	TimeSlot	Y	Enum
startTime	string	Y	HH:mm
district	District	Y	Enum
spaceType	SpaceType	Y	Enum
moods	Mood[]	Y	1~4개
activities	ActivityType[]	Y	1~5개
scheduleDensity	ScheduleDensity	Y	Enum
transportation	Transportation	Y	Enum
initialMessage	string	N	최대 1000자

저장된 사용자 성향을 불러오는 필드는 존재하지 않는다.

처리

1. 현재 ACTIVE 데이트가 있는지 확인
2. 요청 조건 검증
3. 날씨 조회
4. 장소 후보 조회
5. 날씨·거리·활동·분위기·일정 밀도로 후보 점수 계산
6. AI에 후보 장소만 전달
7. 코스 제목, 전체 멘트, 장소별 추천 멘트 생성
8. `ChatSession`, `CourseDraft` 저장

Response 201 data

```
{
  "sessionId": "a68863d8-...",
  "status": "ACTIVE",
  "assistantMessage": {
    "messageId": "e1f4...",
    "role": "ASSISTANT",
    "content": "두 분이 입력한 오늘의 취향과 날씨를 반영해 코스를 만들었어요.",
    "createdAt": "..."
  },
  "courseDraft": {
    "draftId": "c2b7...",
    "version": 1,
    "title": "비 오는 날 포근한 문화 데이트",
    "date": "2026-07-14",
    "timeSlot": "AFTERNOON",
    "overallComment": "오후 비 예보를 반영해 실내 장소 중심으로 구성했어요.",
    "conditions": {},
    "tags": [
      "#유성구",
      "#오후데이트",
    ]
  }
}
```

```

        "#실내데이트",
        "#문화전시",
        "#조용한분위기",
        "#널널한코스"
    ],
    "weather": {},
    "places": [],
    "map": {}
}
}

```

Errors

HTTP	code	설명
409	DATE_COURSE_ACTIVE_ALREADY_EXISTS	현재 데이트가 이미 존재
422	CHAT_NO_RECOMMENDABLE_PLACES	조건에 맞는 장소 부족
422	CHAT_INVALID_DATE_CONDITION	날짜·시간 조건 오류
502	CHAT_AI_PROVIDER_ERROR	AI 응답 오류
503	CHAT_AI_TEMPORARILY_UNAVAILABLE	AI 일시 장애

현재 데이트 취소 기능이 없으므로 ACTIVE 코스가 존재하면 새 코스 생성부터 차단한다.

11.2 채팅 세션 조회

GET /api/v1/chat/sessions/{sessionId}

사용자 검증: 필수, 본인 세션만 가능

Response 200 data

```

{
  "sessionId": "...",
  "status": "ACTIVE",
  "conditions": {},
  "messages": [
    {
      "messageId": "...",
      "role": "USER",
      "content": "마지막은 야경으로 바꿔줘.",
    }
  ]
}

```

```

        "createdAt": "...",
      },
    ],
    "courseDraft": {}
  }

```

11.3 자연어 코스 조정

POST /api/v1/chat/sessions/{sessionId}/messages

사용자 검증: 필수

Request

```

{
  "message": "두 번째 장소를 카페로 바꾸고 이동 거리를 줄여줘.",
  "expectedDraftVersion": 1
}

```

필드	필수	설명
message	Y	1~1000자
expectedDraftVersion	Y	동시 수정 방지

처리 원칙

- AI가 DB에 없는 장소를 생성할 수 없다.
- 장소 교체가 필요하면 Place Repository를 재조회한다.
- 초안 변경 시 version을 1 증가시킨다.
- 수행 불가 시 기존 초안을 유지한다.
- 코스 생성 시 처음 입력한 조건을 기본 컨텍스트로 유지한다.
- 축제 데이터는 검색하거나 반영하지 않는다.

Response 200 data

```

{
  "userMessage": {

```

```

    "messageId": "...",
    "role": "USER",
    "content": "두 번째 장소를 카페로 바꾸고 이동 거리를 줄여
    줘.",
    "createdAt": "..."
  },
  "assistantMessage": {
    "messageId": "...",
    "role": "ASSISTANT",
    "content": "두 번째 장소를 도롱동 카페로 바꾸고 전체 동선을 줄
    였어요.",
    "createdAt": "..."
  },
  "changeSummary": {
    "changed": true,
    "changeTypes": ["PLACE_REPLACED", "ROUTE_OPTIMIZED"],
    "warnings": []
  },
  "courseDraft": {
    "draftId": "...",
    "version": 2
  }
}

```

수행 불가 Response 200

```

{
  "changeSummary": {
    "changed": false,
    "changeTypes": [],
    "warnings": [
      {
        "code": "NO_MATCHING_PLACE",
        "message": "현재 조건과 이동 거리 안에서 대체 가능한 장소
        를 찾지 못했습니다."
      }
    ]
  },
}

```

```
"courseDraft": {}  
}
```

Errors

- CHAT_SESSION_NOT_FOUND
- CHAT_SESSION_FORBIDDEN
- CHAT_SESSION_ALREADY_CONFIRMED
- CHAT_DRAFT_VERSION_CONFLICT
- CHAT_MESSAGE_TOO_LONG

11.4 코스 확정

POST /api/v1/chat/sessions/{sessionId}/confirm

사용자 검증: 필수

Idempotency-Key 권장

Request

```
{  
  "draftId": "c2b7...",  
  "expectedVersion": 2  
}
```

처리

1. ChatSession 소유권 확인
2. ACTIVE 세션 확인
3. Draft ID와 version 확인
4. 현재 ACTIVE 데이트가 없는지 재확인
5. DateCourse 생성
6. DateCoursePlace 스냅샷 일괄 저장
7. ChatSession을 **CONFIRMED** 로 변경

8. 전체 트랜잭션 커밋

Response 201

DateCourseDto

Errors

HTTP	code	설명
409	CHAT_DRAFT_VERSION_CONFLICT	초안 버전 충돌
409	DATE_COURSE_ACTIVE_ALREADY_EXISTS	현재 데이트 존재
404	CHAT_SESSION_NOT_FOUND	세션 없음
409	CHAT_SESSION_ALREADY_CONFIRMED	이미 확정된 세션

기존 현재 코스를 교체하는 옵션은 제공하지 않는다.

11.5 확정 전 초안 폐기

DELETE /api/v1/chat/sessions/{sessionId}

사용자 검증: 필수

현재 데이트가 아니라 확정 전 AI 초안만 폐기한다.

Response 200

```
{
  "success": true,
  "code": "CHAT_SESSION_DISCARDED",
  "message": "코스 초안을 폐기했습니다.",
  "data": null,
  "meta": null,
  "timestamp": "...",
  "traceId": "..."
}
```

▼ 12. Current DateCourse API

12.1 현재 데이트 조회

GET /api/v1/date-courses/current

사용자 검증: 필수

Response 200

DateCourseDto

현재 코스 없음

```
{
  "success": true,
  "code": "DATE_COURSE_CURRENT_EMPTY",
  "message": "진행 중인 데이트 코스가 없습니다.",
  "data": null,
  "meta": null,
  "timestamp": "...",
  "traceId": "..."
}
```

12.2 장소 하트 등록

PUT /api/v1/date-courses/current/places/{coursePlaceId}/heart

사용자 검증: 필수

현재 ACTIVE 코스의 장소에만 사용할 수 있다. 멍등 처리한다.

Response 200 data

```
{
  "coursePlaceId": "...",
  "heartedByMe": true,
  "heartCount": 15,
}
```



```
"updatedAt": "...",
}
```

Errors

- `DATE_COURSE_CURRENT_NOT_FOUND`
- `DATE_COURSE_NOT_ACTIVE`
- `DATE_COURSE_PLACE_NOT_FOUND`
- `DATE_COURSE_PLACE_NOT_OWNED`

12.3 장소 하트 취소

DELETE `/api/v1/date-courses/current/places/{coursePlaceId}/heart`

사용자 검증: 필수

Response 200 data

```
{
  "coursePlaceId": "...",
  "heartedByMe": false,
  "heartCount": 14,
  "updatedAt": "...",
}
```

12.4 데이트 종료

POST `/api/v1/date-courses/current/complete`

사용자 검증: 필수

`Idempotency-Key` 권장

Request

```
{
  "oneLineComment": "비 오는 날에도 이동이 편해서 대화에 집중하기  
좋았어요.",
}
```

```

    "rating": 5
  }

```

필드	타입	필수	검증
oneLineComment	string	Y	1~80자
rating	int	Y	1~5

트랜잭션 처리

1. ACTIVE 코스 조회
2. 소유권 확인
3. 상태를 COMPLETED 로 변경
4. 한 줄 코멘트와 평점 저장
5. 장소별 하트 상태 확정
6. CommunityPost 자동 생성
7. completedAt 기록
8. 전체 커밋

Response 200 data

```

{
  "completedCourse": {},
  "communityPost": {
    "postId": "...",
    "published": true,
    "rankingEligible": true,
    "oneLineComment": "비 오는 날에도 이동이 편해서 대화에 집중하  
기 좋았어요."
  }
}

```

Errors

HTTP	code	설명
404	DATE_COURSE_CURRENT_NOT_FOUND	현재 코스 없음

HTTP	code	설명
409	DATE_COURSE_ALREADY_COMPLETED	이미 종료됨
422	COMMUNITY_COMMENT_REQUIRED	코멘트 없음
500	DATE_COURSE_COMPLETION_TRANSACTION_FAILED	트랜잭션 실패

현재 데이트 취소 API는 제공하지 않는다.

▼ 13. Community / Ranking API

13.1 랭킹보드 목록 조회

GET /api/v1/community/posts

사용자 검증: 선택

Query

필드	타입	설명
<code>sort</code>	RankingSort	기본 <code>OVERALL</code>
<code>district</code>	District[]	다중 선택
<code>timeSlot</code>	TimeSlot[]	다중 선택
<code>spaceType</code>	SpaceType[]	다중 선택
<code>mood</code>	Mood[]	다중 선택
<code>activity</code>	ActivityType[]	다중 선택
<code>density</code>	ScheduleDensity[]	다중 선택
<code>page</code>	int	기본 1
<code>size</code>	int	기본 20, 최대 50

종합 점수 예시

```
overallScore =  
  courseLikeCount * 3  
  + usageCount * 2  
  + placeHeartCount  
  + ratingAverage * 5  
  + freshnessWeight
```

Response 200 data item

```
{  
  "rank": 1,  
  "postId": "...",  
  "courseId": "...",  
  "courseTitle": "도룡동 여유로운 오후 코스",  
}
```

```

    "mainDistrict": "YUSEONG_GU",
    "authorNickname": "대전러버",
    "oneLineComment": "이동 거리가 짧아서 여유롭게 다니기 좋았어  
요.",
    "courseLikeCount": 32,
    "placeHeartCount": 32,
    "usageCount": 128,
    "rating": 4.9,
    "likedByMe": false,
    "tags": ["#유성구", "#오후데이트", "#실내데이트"],
    "publishedAt": "...
  }

```

13.2 랭킹보드 드롭다운 상세 조회

GET /api/v1/community/posts/{postId}

사용자 검증: 선택

Response 200 data

```

{
  "postId": "...",
  "owner": false,
  "courseTitle": "도룡동 여유로운 오후 코스",
  "authorNickname": "대전러버",
  "oneLineComment": "이동 거리가 짧아서 대화하면서 여유롭게 다니  
기 좋았어요.",
  "date": "2026-07-12",
  "timeSlot": "AFTERNOON",
  "overallComment": "실내 문화시설과 야경을 여유롭게 연결했어  
요.",
  "conditions": {},
  "tags": [],
  "places": [],
  "map": {},
  "courseLikeCount": 32,
  "usageCount": 128,
  "rating": 4.9,

```

```
"likedByMe": false,  
"publishedAt": "..."  
}
```

13.3 완료 코스 커뮤니티 재발행

POST /api/v1/community/posts

사용자 검증: 필수

Idempotency-Key 권장

데이트 종료 시 자동 생성되므로 일반 UI에서 직접 호출하지 않는다. 사용자가 커뮤니티 게시물을 삭제한 뒤 과거 코스를 다시 공개할 때 사용한다.

Request

```
{  
  "dateCourseId": "...",  
  "oneLineComment": "이동이 편했어요.",  
  "rating": 5  
}
```

제약

- 본인의 **COMPLETED** 코스만 가능
- 한 완료 코스당 활성 게시물 1개
- 현재 게시물이 존재하면 409

Response 201

커뮤니티 상세 DTO 반환.

Errors

- **COMMUNITY_POST_ALREADY_EXISTS**
- **COMMUNITY_COMPLETED_COURSE_REQUIRED**
- **COMMUNITY_POST_FORBIDDEN**

13.4 한 줄 코멘트 수정

PATCH /api/v1/community/posts/{postId}

사용자 검증: 필수, 소유자만 가능

Request

```
{
  "oneLineComment": "이동이 정말 편했고 비 오는 날에도 좋았어요."
}
```

Response 200

수정된 커뮤니티 상세 DTO 반환.

완료 코스의 장소, 일정, 추천 멘트, 조건은 변경하지 않는다.

13.5 커뮤니티 게시물 삭제

DELETE /api/v1/community/posts/{postId}

사용자 검증: 필수, 소유자만 가능

처리

- 게시물을 **DELETED**
- 랭킹보드 제외
- 과거 데이트 기록 유지
- 기존 좋아요 기록은 통계 보존을 위해 유지

Response 200

data: null

13.6 코스 좋아요 등록

PUT /api/v1/community/posts/{postId}/like

사용자 검증: 필수

한 프로필당 한 게시물에 1회. 멍등 처리.

Response 200 data

```
{
  "postId": "...",
  "likedByMe": true,
  "likeCount": 33
}
```

13.7 코스 좋아요 취소

DELETE /api/v1/community/posts/{postId}/like

사용자 검증: 필수

Response 200 data

```
{
  "postId": "...",
  "likedByMe": false,
  "likeCount": 32
}
```

13.8 공개 코스 진행하기

POST /api/v1/community/posts/{postId}/start

사용자 검증: 필수

Idempotency-Key 권장

Request

```
{
  "date": "2026-07-20",
  "startTime": "14:00"
}
```

처리

1. 현재 ACTIVE 코스 존재 여부 확인
2. 공개 게시물 조회
3. 원본 완료 코스 스냅샷 조회
4. 새 날짜와 시작 시간에 맞춰 일정 시간 재계산
5. 새 ACTIVE 코스 생성
6. `sourceType=RANKING_COPY`
7. 원본 게시물 `usageCount + 1`

Response 201 data

```
{
  "activeCourse": {},
  "copyResult": {
    "sourcePostId": "...",
    "warnings": []
  }
}
```

Errors

HTTP	code
409	<code>DATE_COURSE_ACTIVE_ALREADY_EXISTS</code>
404	<code>COMMUNITY_POST_NOT_FOUND</code>
409	<code>COMMUNITY_POST_DELETED</code>
422	<code>COURSE_COPY_INVALID_START_TIME</code>

현재 데이트 교체 옵션은 존재하지 않는다.

13.9 데이트 마스터 랭킹

GET /api/v1/rankings/masters

사용자 검증: 선택

Query

필드	타입	기본값
<code>period</code>	<code>WEEKLY</code> , <code>MONTHLY</code> , <code>ALL</code>	<code>WEEKLY</code>
<code>limit</code>	int	10, 최대 50

점수 예시

```
masterScore =
  totalCourseUsageCount * 2
+ receivedCourseLikeCount * 3
+ receivedPlaceHeartCount
+ completedCourseCount
```

Response 200 data

```
{
  "period": "WEEKLY",
  "masters": [
    {
      "rank": 1,
      "profileId": "...",
      "nickname": "구름이와몽글이",
      "masterScore": 512,
      "publishedCourseCount": 8,
      "totalUsageCount": 128,
      "receivedLikeCount": 342
    }
  ]
}
```

▼ 14. History API

14.1 과거 데이트 목록 조회

GET /api/v1/profiles/me/date-courses

사용자 검증: 필수

Query

필드	타입	필수
year	int	N
month	int	N
district	District	N
page	int	N
size	int	N

COMPLETED 상태인 본인 코스만 반환한다.

Response 200 data item

```
{
  "courseId": "...",
  "date": "2026-07-12",
  "courseTitle": "도룡동 여유로운 오후 코스",
  "mainDistrict": "YUSEONG_GU",
  "oneLineComment": "이동이 편했어요.",
  "heartedPlaceCount": 3,
  "totalPlaceCount": 3,
  "completedAt": "2026-07-12T20:00:00+09:00"
}
```

14.2 과거 데이트 상세 조회

GET /api/v1/profiles/me/date-courses/{courseId}

사용자 검증: 필수, 본인 코스만 가능

Response 200 data

```
{
  "courseId": "...",
  "status": "COMPLETED",
  "title": "도룡동 여유로운 오후 코스",
  "date": "2026-07-12",
  "oneLineComment": "이동이 편했어요.",
  "rating": 5,
  "conditions": {},
  "tags": [],
  "places": [
    {
      "coursePlaceId": "...",
      "order": 1,
      "scheduledAt": "...",
      "place": {},
      "sweetComment": "...",
      "heartedByMe": true,
      "heartCount": 14
    }
  ],
  "map": {},
  "completedAt": "..."
}
```

14.3 과거 코스 다시 진행하기

POST /api/v1/profiles/me/date-courses/{courseId}/restart

사용자 검증: 필수

Idempotency-Key 권장

Request

```
{
  "date": "2026-07-21",
}
```

```
"startTime": "14:00"  
}
```

처리

1. 현재 ACTIVE 코스 여부 확인
2. 본인 완료 코스 조회
3. 장소·추천 문구·조건 스냅샷 복사
4. 새 날짜와 시작 시간에 맞춰 일정 재계산
5. `sourceType=HISTORY_RESTART`
6. 새 ACTIVE 코스 저장

Response 201

`DateCourseDto`

Errors

- `DATE_COURSE_ACTIVE_ALREADY_EXISTS`
- `HISTORY_COURSE_NOT_FOUND`
- `HISTORY_COURSE_FORBIDDEN`
- `COURSE_COPY_INVALID_START_TIME`

현재 데이트 교체 옵션은 존재하지 않는다.

▼ 15. HTTP 상태 코드 및 예외 처리

15.1 HTTP Status

HTTP	기준
200	조회·수정·삭제 성공, 멍등 재요청
201	새 리소스 생성
400	JSON 파싱 실패, query 조합 오류
401	사용자 검증 헤더 누락 또는 비밀번호 불일치
403	타 사용자 리소스 접근
404	리소스 없음
409	중복·상태·버전 충돌
422	필드 검증 또는 업무 규칙 위반
429	요청 제한
500	내부 서버 오류
502	외부 API 비정상 응답
503	외부 API 일시 장애

15.2 에러 코드

Common

- `COMMON_BAD_REQUEST`
- `COMMON_VALIDATION_ERROR`
- `COMMON_NOT_FOUND`
- `COMMON_INTERNAL_SERVER_ERROR`
- `COMMON_RATE_LIMIT_EXCEEDED`

Auth

- `AUTH_NICKNAME_ALREADY_EXISTS`
- `AUTH_CREDENTIALS_REQUIRED`
- `AUTH_INVALID_PROFILE_ID_FORMAT`
- `AUTH_INVALID_CREDENTIALS`

- AUTH_FORBIDDEN
- AUTH_TOO_MANY_ATTEMPTS

Place

- PLACE_NOT_FOUND
- PLACE_INVALID_COORDINATES
- PLACE_RADIUS_OUT_OF_RANGE

Weather

- WEATHER_DATE_OUT_OF_RANGE
- WEATHER_PROVIDER_ERROR

Chat

- CHAT_SESSION_NOT_FOUND
- CHAT_SESSION_FORBIDDEN
- CHAT_SESSION_ALREADY_CONFIRMED
- CHAT_MESSAGE_TOO_LONG
- CHAT_NO_RECOMMENDABLE_PLACES
- CHAT_INVALID_DATE_CONDITION
- CHAT_DRAFT_VERSION_CONFLICT
- CHAT_AI_PROVIDER_ERROR
- CHAT_AI_TEMPORARILY_UNAVAILABLE

DateCourse

- DATE_COURSE_CURRENT_NOT_FOUND
- DATE_COURSE_ACTIVE_ALREADY_EXISTS
- DATE_COURSE_NOT_ACTIVE
- DATE_COURSE_ALREADY_COMPLETED
- DATE_COURSE_PLACE_NOT_FOUND
- DATE_COURSE_PLACE_NOT_OWNED
- DATE_COURSE_COMPLETION_TRANSACTION_FAILED

Community

- `COMMUNITY_POST_NOT_FOUND`
- `COMMUNITY_POST_FORBIDDEN`
- `COMMUNITY_POST_ALREADY_EXISTS`
- `COMMUNITY_COMPLETED_COURSE_REQUIRED`
- `COMMUNITY_COMMENT_REQUIRED`
- `COMMUNITY_POST_DELETED`

History

- `HISTORY_COURSE_NOT_FOUND`
- `HISTORY_COURSE_FORBIDDEN`

15.3 Validation Error 예시

```
{
  "success": false,
  "code": "COMMON_VALIDATION_ERROR",
  "message": "요청 값이 올바르지 않습니다.",
  "errors": [
    {
      "field": "password",
      "reason": "STRING_PATTERN_MISMATCH",
      "rejectedValue": null
    }
  ],
  "timestamp": "...",
  "traceId": "..."
}
```

비밀번호 오류의 `rejectedValue`는 항상 `null`이다.

15.4 FastAPI 예외 핸들러 대상

- `RequestValidationError`
- `StarletteHTTPException`
- `SQLAlchemyError`

- 외부 API timeout
- 커스텀 `BusinessException`
- 처리되지 않은 `Exception`

```
@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request, exc):
    return JSONResponse(
        status_code=422,
        content=build_error_response(
            code="COMMON_VALIDATION_ERROR",
            message="요청 값이 올바르지 않습니다.",
            errors=[
                {
                    "field": ".".join(map(str, error["loc"]
[1:])),
                    "reason": error["type"],
                    "rejectedValue": None,
                }
                for error in exc.errors()
            ],
            trace_id=request.state.trace_id,
        ),
    )
```

▼ 16. FastAPI 구조

```
app/  
├─ main.py  
├─ common/  
│   ├── dto.py  
│   ├── errors.py  
│   ├── exception_handlers.py  
│   ├── middleware.py  
│   ├── dependencies.py  
│   └─ pagination.py  
├─ auth/  
│   ├── router.py  
│   ├── schemas.py  
│   ├── service.py  
│   ├── repository.py  
│   └─ models.py  
├─ place/  
├─ weather/  
├─ chat/  
├─ course/  
├─ community/  
├─ ranking/  
└─ history/
```

라우터 예시:

```
app.include_router(auth_router, prefix="/api/v1/auth", tags=["Auth"])  
app.include_router(profile_router, prefix="/api/v1/profiles", tags=["Profile"])  
app.include_router(place_router, prefix="/api/v1/places", tags=["Place"])  
app.include_router(weather_router, prefix="/api/v1/weather", tags=["Weather"])  
app.include_router(chat_router, prefix="/api/v1/chat", tags=["Chat"])  
app.include_router(course_router, prefix="/api/v1/date-courses", tags=["DateCourse"])  
app.include_router(communitiy_router, prefix="/api/v1/communitiy", tags=["Community"])  
app.include_router(ranking_router, prefix="/api/v1/rankings", tags=["Ranking"])
```

▼ 17. 페이지별 API 호출 매핑

입장 페이지를 제외한 사용자 페이지에서는 `X-Profile-Id`, `X-User-Password` 를 전달한다.

17.1 시작 페이지

1. `GET /auth/nickname-suggestions`
2. 신규 사용자: `POST /auth/profiles`
3. 기존 사용자: `POST /auth/verify`
4. 입장 후: `GET /profiles/me`

17.2 데이트 코스 만들기

1. `GET /meta/options`
2. 사용자가 매번 전체 성향·조건 입력
3. 선택적으로 `GET /weather`
4. `POST /chat/sessions`
5. `POST /chat/sessions/{sessionId}/messages`
6. `POST /chat/sessions/{sessionId}/confirm`

성향 조회·저장 API는 호출하지 않는다.

17.3 현재 데이트

1. `GET /date-courses/current`
2. `PUT /date-courses/current/places/{coursePlaceId}/heart`
3. `DELETE /date-courses/current/places/{coursePlaceId}/heart`
4. `POST /date-courses/current/complete`

현재 데이트 취소 API는 없다.

17.4 랭킹보드

1. `GET /community/posts`
2. 행 확장: `GET /community/posts/{postId}`
3. `PUT|DELETE /community/posts/{postId}/like`

4. `POST /community/posts/{postId}/start`
5. `GET /rankings/masters`

17.5 과거 데이트

1. `GET /profiles/me/date-courses`
 2. 행 확장: `GET /profiles/me/date-courses/{courseId}`
 3. `POST /profiles/me/date-courses/{courseId}/restart`
-

▼ 18. Vue API Client 기준

18.1 저장 정책

```
sessionStorage.setItem("sai42.profileId", response.data.profileId);  
sessionStorage.setItem("sai42.password", enteredPassword);
```

- `localStorage` 사용 금지
- 탭 종료 시 자동 삭제
- 퇴장 버튼은 두 값을 삭제하고 시작 페이지로 이동
- 로그아웃 API 호출 없음

18.2 Fetch Wrapper

```
export async function apiFetch(path, options = {}) {  
  const profileId = sessionStorage.getItem("sai42.profileId");  
  const password = sessionStorage.getItem("sai42.password");  
  
  const headers = new Headers(options.headers || {});  
  headers.set("Content-Type", "application/json");  
  
  if (profileId && password) {  
    headers.set("X-Profile-Id", profileId);  
    headers.set("X-User-Password", password);  
  }  
  
  const response = await fetch(`${API_BASE_URL}${path}`, {  
    ...options,  
    headers,  
  });  
  
  const body = await response.json();
```

```

if (response.status === 401) {
    sessionStorage.removeItem("sai42.profileId");
    sessionStorage.removeItem("sai42.password");
}

return body;
}

```

▼ 19. 개발 완료 기준

1. 모든 성공 응답이 `BaseDto` 구조를 따른다.
2. 모든 실패 응답이 `ErrorResponseDto` 구조를 따른다.
3. FastAPI 기본 422 응답을 공통 `ErrorResponseDto`로 변환한다.
4. 보호 API는 매 요청마다 `profileId`와 평문 비밀번호를 검증한다.
5. 비밀번호가 로그·에러 응답·trace에 노출되지 않는다.
6. 닉네임과 비밀번호 수정 API가 존재하지 않는다.
7. 사용자 성향은 프로필에 저장하지 않는다.
8. AI 코스 생성 요청마다 전체 성향 조건을 필수 전달한다.
9. 축제 조회 및 축제 기반 추천 로직이 존재하지 않는다.
10. 한 프로필당 ACTIVE 코스가 하나만 존재한다.
11. ACTIVE 코스가 존재하면 새 코스 생성·랭킹 복사·과거 재진행을 409로 차단한다.
12. 현재 데이트 취소 API가 존재하지 않는다.
13. 장소 하트는 현재 ACTIVE 코스에서만 변경할 수 있다.
14. 데이트 종료와 커뮤니티 자동 공개를 하나의 트랜잭션으로 처리한다.
15. 완료 코스의 장소·시간·추천 문구는 변경되지 않는다.
16. 커뮤니티 수정은 한 줄 코멘트만 허용한다.
17. AI가 DB에 존재하지 않는 장소를 코스에 등록하지 못한다.
18. 날씨 API 실패가 코스 생성 전체 실패로 이어지지 않는다.
19. 완료 코스 스냅샷은 원본 공공데이터가 변경되어도 유지된다.
20. `/docs`, `/redoc`에서 모든 Request/Response/Error Schema를 확인할 수 있다.